Bharat Ratna Indira Gandhi College of Engineering, Solapur

Master of Computer Application

Subject: Java Programming

UNIT-I

Prepared By

Asst.Professor. Jagdish. D. Patil (MCA).

❖ What is Java?

Developed by Sun Microsystems in 1995, Java is a highly popular, object-oriented programming language. This platform independent programming language is utilized for Android development, web development, artificial intelligence, cloud applications, and much more.

❖ Feature of Java:

1. Platform independent language:
Java is guaranteed to be write-once, run-anywhere language.
On compilation Java program is compiled into bytecode. This bytecode is platform independent and can be run on any machine.

2. Simple and easy to create:
Java is easy to learn and its syntax is quite simple, clean and easy to understand.

3. Object oriented programming language:
In java, everything is an object which has some data and behavior.

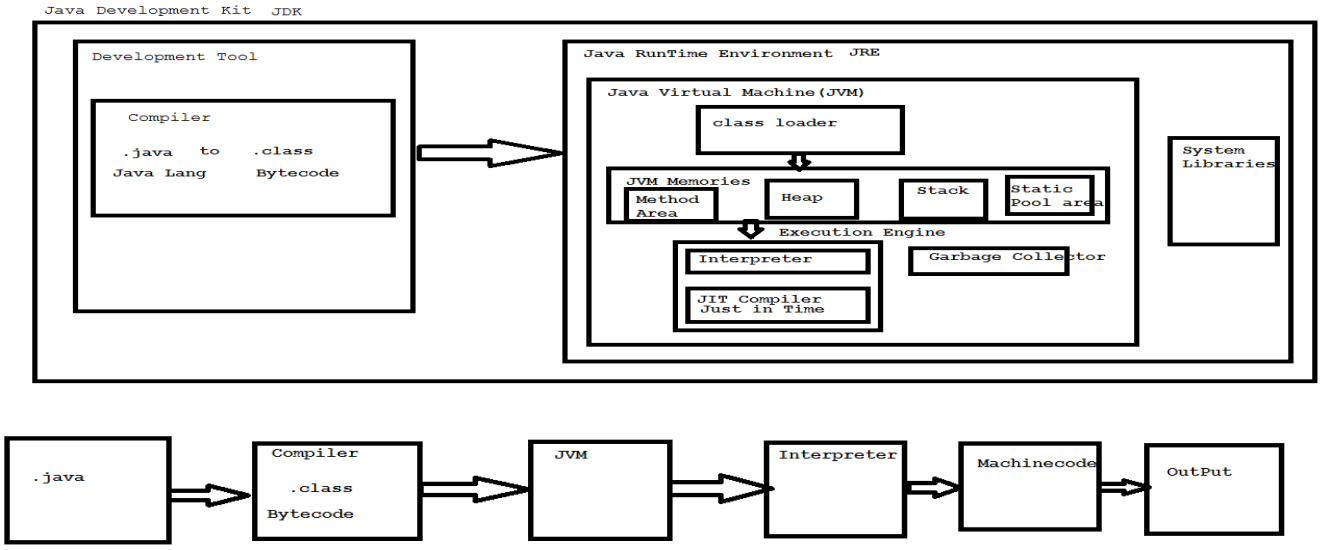
4. Secure:
Java secure features it enables us to develop virus free, temper free system.

5. Portable:
Java Byte code can be carried to any platform.

6. Multithreading:
Java multithreading feature makes it possible to write program that can do many tasks simultaneously.

7. Garbage collector:
The garbage collector finds unused objects and deletes them to free up memory.

❖ Architecture of Java:



- **Java Development Kit:**

JDK (Java Development Kit) is a software development kit required to develop applications in Java. When you download JDK, JRE is also downloaded with it.

In addition to JRE, JDK also contains a number of development tools (compilers, JavaDoc, Java Debugger, etc.).

- **Compiler:**

A compiler in Java is a computer program that is used for compiling Java programs. It is platform-independent. It converts (translates) source code (.java file) into bytecode (.class file).

- **Java Runtime Environment:**

JRE (Java Runtime Environment) is a software package that provides system libraries, Java Virtual Machine (JVM), and other components that are required to run Java applications.

Java Virtual Machine:

It is Runtime Engine responsible to run Java programs.

It has three main parts;

1. Class Loader:

- It reads .class file and store corresponding information into JVM memories.

2. JVM memories:
- It divides into four parts:

A. Method area: It stores .class file information and variables
B. Heap Area: It stores objects, Arrays, non static method declaration.
C. Stack: main method execution flow
D. Static pool area: It stores static method declaration.

3. Execution Engine:
- It is responsible for executing java class file.
- It contains two components:

A. Interpreter:
- Interpreter is basically a translator which converts Java Bytecodes into machine code (the actual code understandable to computer known as binary code). it translate bytecodes line by line.
- The need of Interpreter is to achieve platform Independency and Security of program from virus.

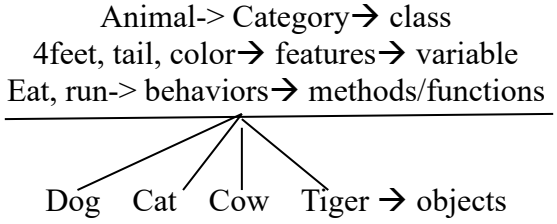
B. JIT(Just in time) compiler:
- The JIT Compiler speeds up the performance of the Java applications at run time.
- **JIT** stands for **Java-In-Time Compiler**. The **JIT compilation** is also known as dynamic compilation.
- If the JIT compiler environment variable is properly set, the JVM reads the .class file (bytecode) for interpretation after that it passes to the JIT compiler for further process. After getting the bytecode, the JIT compiler transforms it into the native code (machine-readable code).

Steps to create Java Project in Eclipse:

Step1: create java project--> file--> new--> java project--> enter project name--> finish

Step2: create java package--> right click on project name/Src folder-->new--> package--> enter package name--> finish

Step3: create java class-->right click on package name-->new-->class-->enter class name--> finish.



❖ What is Class?

- Class is collection of objects, variables and methods.
- Classname should be in Camelcase→ HelloWorldProgram, Create_Login_Page
- Class is plan through which we come to know about different logics we are having on our program.

Class-→predefined &user defined
Predefined-→scanner, console, System, String
User defined-→Hello, Test, Demo etc.

Syntax:

```
Access_specifier class classname
{
        //variables
        //methods
        //objects          }
```

❖ First Java Program:

Steps to Follow:

1. Create java project
2. create java package
3. create java class
4. main method
5. printing statement
6. save program (Control + s)
7. run program (click green button)
8. check output--> Console tab

```
package FirstProgram; //within 1 package multiple classes are allowed

public class HelloWorldProgram  //defined class
{
        //starting class body

        public static void main(String[] args) //main method declaration
        {



                System.out.print("Hello World");
```

```
	}
}
```

```
package FirstProgram;

public class DisplayInformation
{
	//Program to display information
	public static void main(String[] args)
	{
		System.out.println("Jagdish Patil");
		System.out.println("Solapur");
		System.out.println("BIGCE");
	}
}
```

❖ Variables:

Variables are nothing but piece of memory use to store information.
one variable can store 1 information at a time.
Variables also used in information reusability.
To utilize variables in java programming language we need to follow below steps:
1. Variable declaration (Allocating/Reserving memory)
2. Variable Initialization (Assigning or inserting value)
3. Usage

Rules to Declare a Variable

- A variable name can consist of Capital letters A-Z, lowercase letters a-z digits 0-9, and two special characters such as _ underscore and $ dollar sign.
- The first character must not be a digit.
- Blank spaces cannot be used in variable names.
- Java keywords cannot be used as variable names.
- Variable names are case-sensitive.
- There is no limit on the length of a variable name but by convention, it should be between 4 to 15 chars.
- Variable names always should exist on the left-hand side of assignment operators.

Types of variables-→

Global variable

- These are variables which you can declare outside method/constructor within class.

1. **Static variable**

- It is class level variables
- memory allocate at the time of class loading
- memory deallocate at the time of class unloading
- Static variable are accessible throughout class in any method without creation of object.
- If we do not assign any value to static variable then default values get assigned to that variable.
- Static variable value changes happen throughout class.

- Syntax:

static datatype variablename= value;

Example:

```
static int a=20;
static float d=3.4f;


package variableexamples;


public class demo1
{
   static boolean a =true; //static global variable-->declare on class level
        public static void main(String[] args)
        {
                System.out.println(a);
                //a=20;
                m1();
        }
        public static void m1()
        {
```

```
            System.out.println(a);
        }
}
```

2. Non-static variable/instance

- Defined at class level but can be accessed only through an object.
- Non static variable value can be changed from object to object.
- If we do not assign any value to non static variable then default values get assigned to that variable.
- To call non static variable inside static method object creation is required.
- To call non static variable inside non static method object creation is not required.

Syntax:
Datatype variablename= value;

Example.

```
package variableexamples;


public class demo2
{
        int b=20;  //non static global variable
        public static void main(String[] args)
        {
                demo2 obj= new demo2();
                System.out.println("1."+obj.b);//20
                obj.b=40;
                System.out.println("2"+obj.b);//40
                obj.m2();
                m3();
                demo2 obj1= new demo2();
                System.out.println(obj1.b); //20
                obj1.b=36;
                System.out.println(obj1.b);  //36
```

```
    }
    public void m2()
    {
        b=50;
        System.out.println("3."+b);//50
    }
    public static void m3()
    {
        demo2 obj= new demo2();
        System.out.println("4."+obj.b);//20
        obj.b=60;
        System.out.println("5."+obj.b);//60
    }
}
```

3. Local variable

- Local variables are variables which are created within method/constructor.
- Scope of local variable remains only within that method/constructor.
- Temporary variables
- Default values are not provided for local variables.

Example:

```
package variableexamples;
public class local_variable
{
    static int d=30;
    public void m1()
    {
        int c=30;//local variable-->temporary variables
        System.out.println(c);
    }
```

```
        public static void m2()
        {
                int c=10;
                System.out.println(c);
        }
        public static void main(String[] args)
        {
                m2();
                local_variable obj= new local_variable();
obj.m1(); } }
```

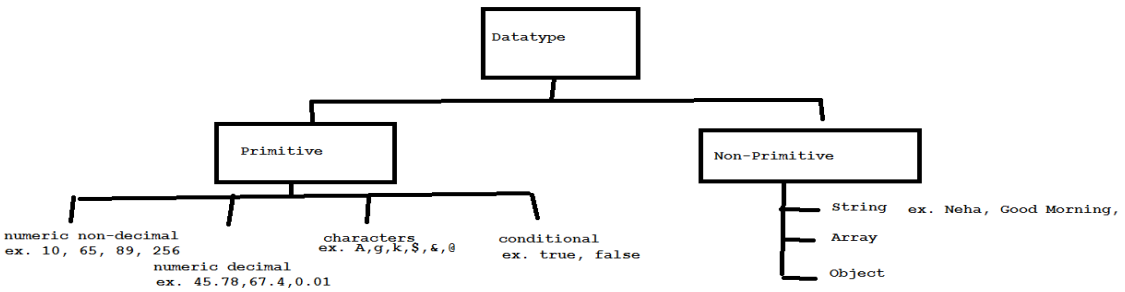
❖ Data Types:

Data types are used to represent type of data or information which we are going to use in our java program.

In java programming it is mandatory to declare data type before declaration of variable.

In java datatypes are classified into two types:

1. Primitive datatype.
2. Non-primitive datatype.



1.Primitive datatype:
There are 8 types of primitive datatypes.
-all the primitive datatypes are keywords.
-Memory size of primitive datatype is fix.
-The types of primitive datatype are:

syntax:

datatype variablename;

DataType	Size
Byte	1 byte=8bit
Short	2 bytes =16 bit
Int	4 bytes =32bit
Long	8 bytes=64bit
Float	4 byte=32bit
double	8 byte
Char	2 byte
Boolean	1 bit

2.Non-primitive datatype:

There are 3 types of non-primitive datatypes.

all the Non primitive datatypes are identifiers.

* Memory size of non-primitive datatype is not defined or not fix.

Non-primitive datatype starts with capital letter.

Primitive Datatype	Non-primitive Datatype
1. keywords-reserved words	1. identifiers
2. memory fix	2. memory size not fix
3. predefined by Java	3. created by programmers
4. always needs value	4. value null
5. in lowercase format	5. first letter is in uppercase format
6. int,float,double, char	6. String, Array, object

❖ Operators:

Operator in Java is a symbol that is used to perform operations on Variables and values.
Example:

int n1=10;

int n2=20;

+ → operator

n1+n2 -> 10+20 ->30

Syntax:

Operand1 operator operand2

Operand operator

Operator operand

1. Arithmetic Operator:
Arithmetic operators are used to perform arithmetic operations on variables and data.

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo Operation (Remainder after division)

Example:

```
package operators;

public class ArithmeticOperators
{

	public static void main(String[] args)
	{
		int num1=30;//operand1
```

```
        int num2=20;//operand2

        // +  -> Addition operation
        System.out.println(num1+num2);//30+20->50

        // -  -> Substraction operation
        System.out.println(num1-num2);//30-20->10

        // *  -> Multiplication
        System.out.println(num1*num2);//30*20->600

        // /  -> Division
        System.out.println(num1/num2);//30/20->1.5

        // %  ->Modulous
        System.out.println(num1%num2);//30%20->10
    }

}
```

2. Assignment operators:

Assignment operators are used in Java to assign values to variables.

Operator	Example	Equivalent to
=	a = b;	a = b;
+=	a += b;	a = a + b;
-=	a -= b;	a = a - b;
*=	a *= b;	a = a * b;
/=	a /= b;	a = a / b;
%=	a %= b;	a = a % b;

Example:

```
package operators;

public class assignmentoperator {

	public static void main(String[] args)
	{
		//equal to operator-> =
		int n1=20;
		int n2=40;

		System.out.println(n1);

		// += operator

		System.out.println(n1+=n2);//n1=n1+n2  n1=20+40  n1=60
		System.out.println("Value of n1 after += is "+n1);

		// -=
		System.out.println(n1-=n2);//n1=n1-n2  n1=60-40  n1=20
		System.out.println("Value of n1 after -= is "+n1);

		//*=

		System.out.println(n1*=n2);//n1=n1*n2 n1=20*40 n1=800
		System.out.println("Value of n1 after *= is "+n1); } }
```

3. Relational operators:

Relational operators are used to check the relationship between two operands.

Operator	Description	Example
==	Is Equal To	3 == 3 returns **true**
!=	Not Equal To	3 != 4 returns **true**
>	Greater Than	1 > 2 returns **false**
<	Less Than	1 < 3 returns **true**

>=	Greater Than or Equal To	3 >= 1 returns **true** **3>1 or 3==1→true** **3>=3** **3>3 or 3==3→ true** **2>=4 return false**
<=	Less Than or Equal To	3 <= 2 returns **false** **3<2or 3==2** **2<=2 return true**

Example:

```
package operators;
public class Relational
{
	public static void main(String[] args)
	{
		int a=10, b=20;
		System.out.println("Value of a= "+a+" Value of b= "+b);
		//  == operator
		System.out.println("Condition is "+(a==b));// 10==20  false
		//  != operator
		System.out.println("Condition is "+(a!=b));// 10!=20  true
		//  > operator
		System.out.println("Condition is "+(a>b));// 10>20 false
		// < operator
		System.out.println("Condition is "+(a<b));//  10<20 true
		// >= operator
		System.out.println("Condition is "+(a>=b));//  10>20 or 10==20 false
		//<= operator
		System.out.println("Condition is "+(a<=b));//  10<20 or 10==20 true
	}
}
```

4. Logical operators:

Logical operators are used to check whether an expression is true or false.

Operator	Example	Meaning
&& (Logical AND)	expression1 **&&** expression2	true only if both expression1 and expression2 are true
|| (Logical OR)	expression1 **||** expression2	true if either expression1 or expression2 is true
! (Logical NOT)	**!**expression	true if expression is false and vice versa

AND-> &&

Expression1 && Expression2			
TRUE	&&	TRUE =>	TRUE
TRUE	&&	FALSE➔	FALSE
FALSE	&&	TRUE =>	FALSE
FALSE	&&	FALSE =>	FALSE

OR-> ||

Expression1 || Expression2			
TRUE	||	TRUE =>	TRUE
TRUE	||	FALSE➔	TRUE
FALSE	||	TRUE =>	TRUE
FALSE	||	FALSE =>	FALSE

NOT-> !

!expression1-> !TRUE➔False		
!false➔ true		
TRUE	!	->FALSE
FALSE	!	->TRUE

Example:

```
package operators;
public class Logicaloperator
{
	public static void main(String[] args)
	{
		int a=10, b=20, c=30;
		//&& operator
		System.out.println((a<b)&&(b<c));
		System.out.println((a<b)&&(b>c));
		System.out.println((a>b)&&(c>b));
		System.out.println((a>b)&&(b>c));
		// || operator
		System.out.println("**************************");
		System.out.println((a<b)||(b<c));
		System.out.println((a<b)||(b>c));
		System.out.println((a>b)||(c>b));
		System.out.println((a>b)||(b>c));
		// ! operator
		System.out.println("**************************");
		System.out.println(!(a<b));
		System.out.println(!(a>b));

	}
}
```

5. Unary operators:
Unary operators are used with only one operand.

Operator	Meaning
++	Increment operator: increments value by 1 -> a++-> a+1 ex. A=4 A++ A+1=> 4+1 => 5
--	Decrement operator: decrements value by 1 -> a-- -> a-1 ex. A=4 A-- A-1 => 4-1 => 3

Post increment

A++ => A A+1→ a=a+1

Post Decrement
A-- => A A-1 → a=a-1

Pre increment
++A => A+1 A

Pre Decrement
--A => A-1 A

Example:

```
package operators;

public class Unaryoperator
{
     public static void main(String[] args)
     {
          int a=10, b=20;

          //Postincrement

System.out.println("using postincrement operator "+(a++)); //10
System.out.println("After postincrement value of a "+a);   //11

          //Postdecrement
System.out.println("using postdecrement operator "+(b--)); //20
System.out.println("After postdecrement value of b "+b);   //19
```

```
          //Preincrement
System.out.println("using preincrement operator "+(++a)); //12
System.out.println("a= "+a);  //12

          //Predecrement
System.out.println("using predecrement operator "+(--b)); //18
System.out.println("b= "+b); //18
     }}
```

❖ What are Strings in Java?

Strings are the type of objects that can store the character of values and in Java, every character is stored in 16 bits i.e. using UTF 16-bit encoding. A string acts the same as an array of characters in Java.

Ways of Creating a String

There are two ways to create a string in Java:

- String Literal
- Using new Keyword

1. String literal

To make Java more memory efficient (because no new objects are created if it exists already in the string constant pool).

String demoString = “HelloJava”;

2. By new keyword

String s=**new** String("Welcome");//creates two objects and one reference variable

Example:

```
public class StringExample{
public static void main(String args[]){
String s1="java";//creating string by Java string literal
char ch[]={'s','t','r','i','n','g','s'};
String s2=new String(ch);//converting char array to string
```

```
String s3=new String("example");//creating Java string by new keyword
System.out.println(s1);
System.out.println(s2);
System.out.println(s3);
}}
```

String Methods:

1. int length()

Returns the number of characters in the String.

"HelloJava".length(); // 9

2. Char charAt(int i)

Returns the character at ith index.

"HelloJava".charAt(3);

3. **String substring (int i)**

Return the substring from the ith index character to end.

"HelloJava".substring(3);

4. String substring (int i, int j)

Returns the substring from i to j-1 index.

"HelloJava".substring(2, 5);

5. String concat(String str)

Concatenates specified string to the end of this string.

String s1 = ”Hello”;

String s2 = ”Java”;

String output = s1.concat(s2); // returns “HelloJava”

6. **int indexOf (String s)**

Returns the index within the string of the first occurrence of the specified string.

If String s is not present in input string then -1 is returned as the default value.

1. String s = ”Learn Share Learn”;

   int output = s.indexOf(“Share”); // returns 6

7. int indexOf (String s, int i)

Returns the index within the string of the first occurrence of the specified string, starting at the specified index.

String s = ”Learn Share Learn”;

int output = s.indexOf("ea",3);// returns 13

8. Int lastIndexOf(String s)

Returns the index within the string of the last occurrence of the specified string.

If String s is not present in input string then -1 is returned as the default value.

1. String s = ”Learn Share Learn”;

   int output = s.lastIndexOf("a"); // returns 14

2. String s = "Learn Share Learn"

   int output = s.indexOf(“Play”); // return -1

9. boolean equals(Object otherObj)

Compares this string to the specified object.

Boolean out = “Hello”.equals(“Hello”); // returns true

Boolean out = “Java”.equals(“java”); // returns false

10. boolean equalsIgnoreCase (String anotherString)

Compares string to another string, ignoring case considerations.

Boolean out= “Hello”.equalsIgnoreCase(“Hello”); // returns true

Boolean out = “Hello”.equalsIgnoreCase(“hello”); // returns true

11. int compareTo(String anotherString)

Compares two string lexicographically.

int out = s1.compareTo(s2);

// where s1 and s2 are

// strings to be compared

This returns difference s1-s2. If :

out < 0 // s1 comes before s2

out = 0 // s1 and s2 are equal.

out > 0 // s1 comes after s2.

12. int compareToIgnoreCase(String anotherString)

Compares two string lexicographically, ignoring case considerations.

int out = s1. compareToIgnoreCase(s2);

// where s1 and s2 are

// strings to be compared

This returns difference s1-s2. If :

out < 0 // s1 comes before s2

out = 0 // s1 and s2 are equal.

out > 0 // s1 comes after s2.

Note: In this case, it will not consider case of a letter (it will ignore whether it is uppercase or lowercase).

13. String toLowerCase()

Converts all the characters in the String to lower case.

String word1 = “HeLLo”;

String word3 = word1.toLowerCase(); // returns “hello"

14. String toUpperCase()

Converts all the characters in the String to upper case.

String word1 = “HeLLo”;

String word2 = word1.toUpperCase(); // returns “HELLO”

15. String trim()

Returns the copy of the String, by removing whitespaces at both ends. It does not affect whitespaces in the middle.

String word1 = “ Learn Share Learn “;

String word2 = word1.trim(); // returns “Learn Share Learn”

16. String replace (char oldChar, char newChar)

Returns new string by replacing all occurrences of oldChar with newChar.

String s1 = “Hello“;

String s2 = “Hello”.replace(‘f’ ,’s’); // return “fells”

Note: s1 is still Hello and s2 is fells

17. boolean contains(CharSequence sequence)

Returns true if string contains contains the given string

String s1="HelloJava";

String s2="Java";

s1.contains(s2) // return true

18. Char[] toCharArray():

Converts this String to a new character array.

String s1="HelloJava";

char []ch=s1.toCharArray(); //return [‘H’,’e’,……’a’]

19. boolean startsWith(String prefix)

Return true if string starts with this prefix.

String s1="HelloJava";

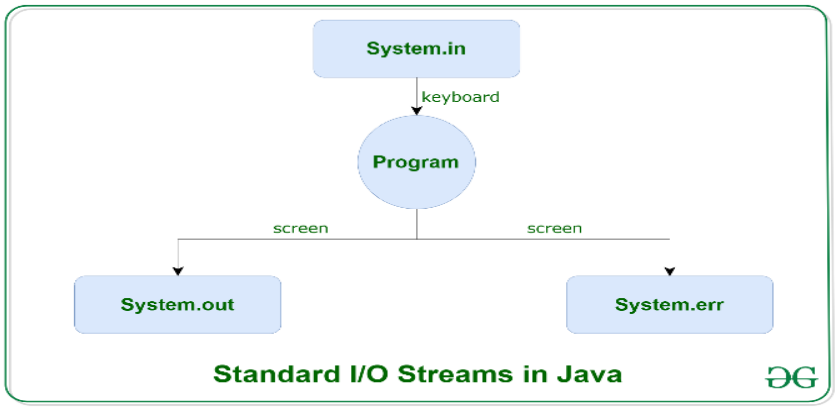
String s2="H";

s1.startsWith(s2) // return true

❖ Input Output Operations:

Java brings various Streams with its I/O package that helps the user to perform all the input-output operations. These streams support all the types of objects, data-types, characters, files etc to fully execute the I/O operations.

Before exploring various input and output streams let's look at 3 standard or default streams that Java has to provide which are also most common in use:



System.in: This is the standard input stream that is used to read characters from the keyboard or any other standard input device.

System. Out: This is the standard output stream that is used to produce the result of a program on an output device like the computer screen.

Here is a list of the various print functions that we use to output statements:

Print();

Println ();

System.err: This is the **standard error stream** that is used to output all the error data that a program might throw, on a computer screen or any standard output device.

This stream also uses all the 3 above-mentioned functions to output the error data:

- print()
- println()
- printf()

❖ Arrays in Java

- In Java, all arrays are dynamically allocated.
- Arrays may be stored in **contiguous memory** [consecutive memory locations].
- Since arrays are objects in Java, we can find their length using the object property *length*. This is different from C/C++, where we find length using size of.
- A Java array variable can also be declared like other variables with [] after the data type.
- The variables in the array are ordered, and each has an index beginning with 0.
- Java array can also be used as a static field, a local variable, or a method parameter.

Creating, Initializing, and Accessing an Arrays in Java

The general form of array declaration is

-- type var-name [];
-- type [] var-name;

- An array declaration has **two components**: the type and the name.
- *type* declares the element type of the array.
- The element type determines the data type of each element that comprises the array.
- Like an array of integers, we can also create an array of other primitive data types like char, float, double, etc., or user-defined data types (objects of a class).
- Thus, the element type for the array determines what type of data the array will hold.

Instantiating an Array in Java

When an array is declared, only a reference of an array is created. To create or give memory to the array, you create an array like this: The general form of *new* as it applies to one-dimensional arrays appears as follows:

var-name = new type [size];

Types of Arrays in Java

Java supports different types of arrays:

1. **Single-Dimensional Arrays**

These are the most common type of arrays, where elements are stored in a linear order.

Syntax: dataType[] arr;

Example:

```
class Testarray{
public static void main(String args[]){
int a[]=new int[5];//declaration and instantiation
a[0]=10;//initialization
a[1]=20;
a[2]=70;
a[3]=40;
a[4]=50;
//traversing array
for(int i=0;i<a.length;i++)//length is the property of array
System.out.println(a[i]);
}}
```

2. **Multi-Dimensional Arrays**

Arrays with more than one dimension, such as two-dimensional arrays (matrices).

int[][] arr=**new int**[3][3];//3 row and 3 column

Example:

```
class Testarray3{
public static void main(String args[]){
//declaring and initializing 2D array
int arr[][]={{1,2,3},{2,4,5},{4,4,5}};
```

```
//printing 2D array
for(int i=0;i<3;i++){
 for(int j=0;j<3;j++){
   System.out.print(arr[i][j]+" ");
 }
 System.out.println();
}
}}
```

3. String Arrays:

Arrays can also hold elements of reference types, such as objects.

String arrays are arrays of strings, which are objects in Java.

Example:

```
String[] colors = {"red", "green", "blue"};
String favoriteColor = colors[0]; // Accessing the first element (value: "red")
```

4. Arrays of Objects:

You can create arrays of any user-defined class or object.

This allows you to store collections of custom objects in an array.

Example:

```
class Person {
 String name;
 int age;
}
Person[] people = new Person[3]; // Create an array of Person objects
people[0] = new Person();
people[0].name = "Alice";
people[0].age = 30;
```